

Reviewing, Validating, and Maintaining Agent Output

Automated gates catch what they're built to catch, not everything.

What You'll Learn

1

Apply human-in-the-loop review even after code has cleared the checklist and analyze gates

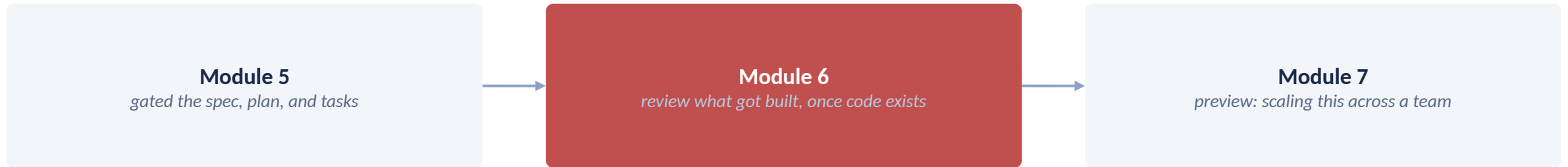
2

Recognize hallucinations, unsafe changes, and weak assumptions that automated gates won't catch

3

Manage spec drift so the spec, constitution, and agent instruction files stay accurate as an audit trail

Passing Every Gate Is Not the Same as Being Correct



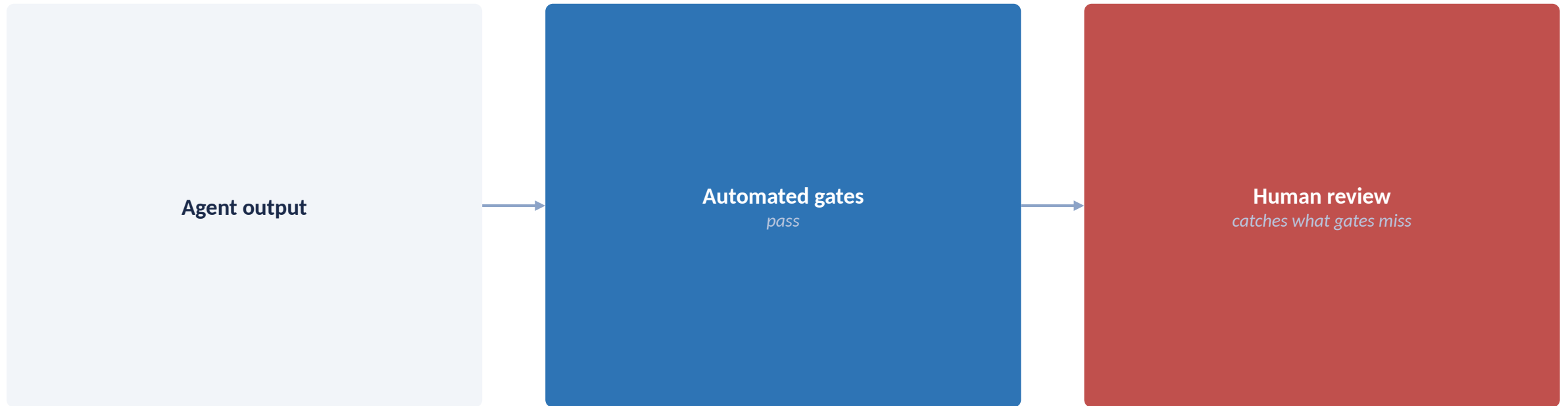
Module 5's gates run before code exists and check the artifacts. Once an agent writes code, a new class of problem becomes possible: output that clears every automated check and is still wrong.

PART 1 • AFTER THE GATES

Human-in-the-Loop Still Matters

What to verify before accepting implemented code, even after it clears automated checks

Human-in-the-Loop Still Matters



Passing every gate is not the same as being correct.

What to Verify, Even After the Gates Pass

Does it match the spec, not just the tests?

Tests can pass while quietly narrowing scope, an agent can satisfy the letter of a requirement while missing its intent. Re-read the spec against the diff, not just the test output.

Is the change scoped to what was asked?

Unrelated edits, deleted comments, or 'while I was in there' refactors are a common failure mode. A clean diff review catches scope creep a passing test suite won't.

Would you be comfortable explaining this code to someone else?

If a reviewer can't explain why the implementation works, that's a signal to slow down, not a formality to skip because the gates already passed.

Added Scrutiny for Production Systems

Blast radius

What breaks if this is wrong, and who notices first: an internal tool, a customer-facing flow, or a system another team depends on? Higher blast radius earns a slower, more deliberate review.

Rollback and monitoring

Before accepting, confirm there's a fast way to undo the change and a way to know quickly if it's causing problems. If either is missing, that's a gap to close before merge, not after.

Staged exposure

For production-critical paths, prefer a staged rollout (feature flag, canary, percentage rollout) over a single all-at-once deploy, even when every gate passed.

Checkpoint: Why review code that already passed every gate?

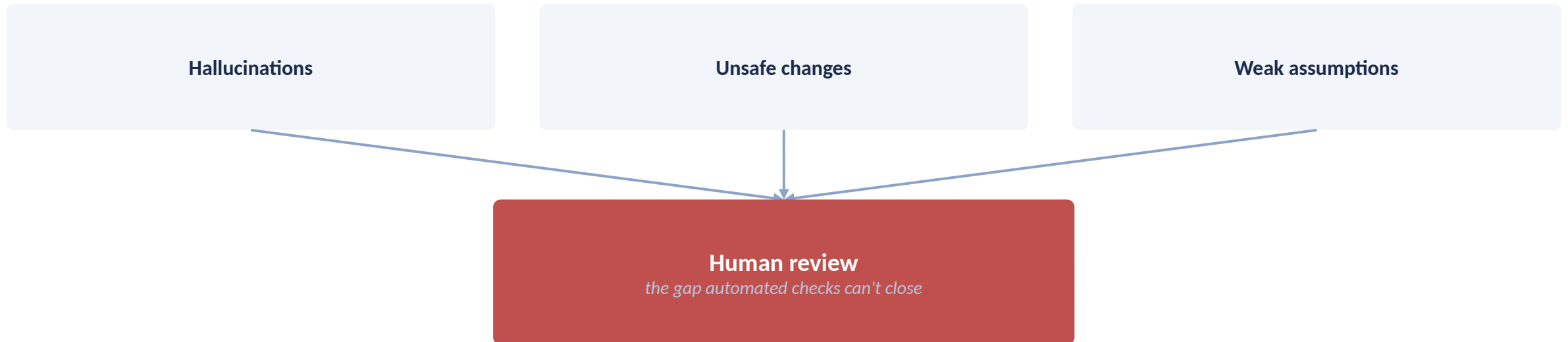
- Gates check what they were built to check; intent, judgment, and scope aren't fully covered by automated checks.
- A clean diff review catches scope creep that a passing test suite won't.
- Review depth should scale with blast radius: production-critical changes earn more scrutiny, not the same fixed pass every time.

PART 2 • WHAT GATES WON'T CATCH

Hallucinations, Unsafe Changes, and Weak Assumptions

Three failure modes that look like working code

The Riskiest Changes Are the Ones That Look Done



Confident, polished output is not the same as output that actually matches the spec.

Module 6 at a Glance

	GitHub Spec Kit	OpenSpec
Post-implementation review	Re-run /speckit.analyze; update constitution/spec	/opsx:verify (Completeness/Correctness/Coherence)
Keeping artifacts in sync	Update constitution, spec, or clarify answers	/opsx:update
Shared goal	Confirm what got built still matches what was decided	Same

CHECKPOINT

Checkpoint: What do hallucinations, unsafe changes, and weak assumptions have in common?

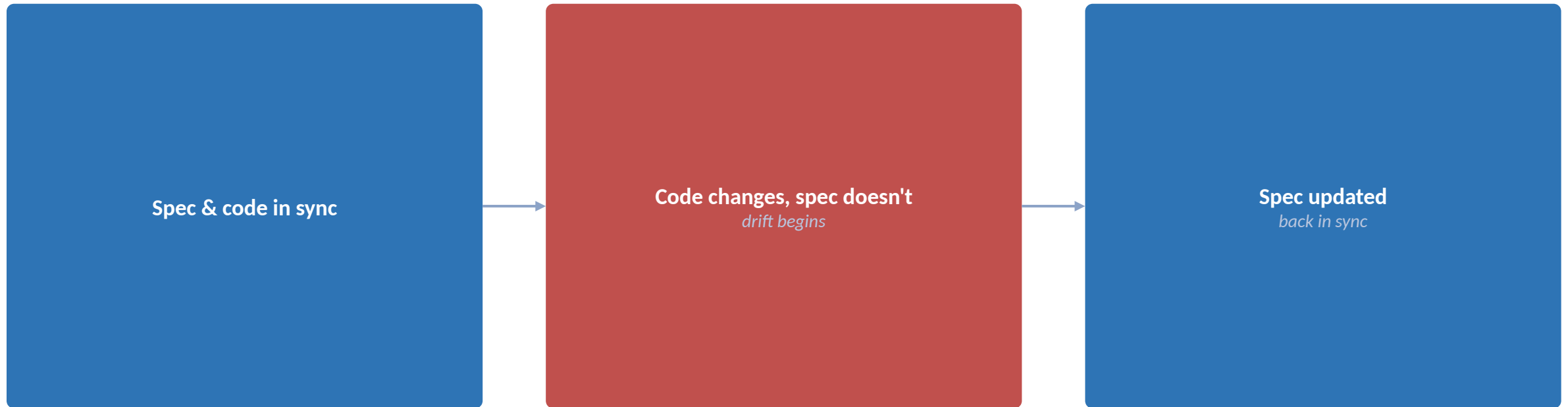
- All three can produce code that runs, passes tests, and still doesn't do the right thing.
- None of them reliably trigger an automated gate, because the code often looks correct on its face.
- OpenSpec's `/opsx:verify` and Spec Kit's `re-run /speckit.analyze` both exist to catch exactly this, after code exists.

PART 3 • STAYING CURRENT

Managing Spec Drift

Keeping the spec, constitution, and agent instruction files true as the codebase evolves

Managing Spec Drift



Left unmanaged, documentation quietly stops being true.

Keeping Specs and Constitution Current

```
/speckit.analyze
```

Re-run after implementation as an extra review pass, not just before, to catch drift between what was built and what was specified, in case the implementation quietly diverged from the plan along the way.

Closing the gap

When review surfaces a gap, update the constitution or the spec directly (via `/speckit.specify` or another `/speckit.clarify` pass) rather than letting the code silently diverge from them, so the documentation stays the thing you can actually trust.

```
/speckit.converge
```

Assesses the codebase against the feature's spec, plan, and tasks and appends any missed work as new tasks. Append-only: it never edits or deletes code directly, so it's a safe way to surface gaps without risking an unreviewed change.

Checkpoint: Why treat spec drift as an audit-trail problem, not just a tidiness problem?

- A spec that no longer matches the code can't be trusted the next time someone, or some agent, reads it to make a decision.
- `/speckit.converge` and `/opsx:update` both make staying in sync part of the normal workflow, not a separate cleanup task.
- The goal isn't a perfect document, it's documentation that's accurate enough to build on.

Common Pitfalls

Trusting a green checkmark as the finish line

Automated gates passing is necessary, not sufficient. Skipping human review because 'the tests passed' is how confident, wrong code ships.

Applying the same review depth to every change

A one-line config tweak and a production auth change don't warrant the same scrutiny. Fixed-depth review either wastes time or misses risk.

Letting spec updates lag behind shipped code

Treating documentation sync as optional cleanup means the next person, human or agent, builds on a spec that's already wrong.

Key Takeaways

- Passing every gate is not the same as being correct, which is why human review still has a job to do afterward.
- Hallucinations, unsafe changes, and weak assumptions can all produce code that looks done without being correct; review depth should scale with what's at stake.
- Both frameworks converge on the same post-implementation question: does what got built still match what was decided, and if not, why not?
- Keeping documentation current is what makes it worth trusting the next time someone, or some agent, reads it. Module 7 picks up from here: scaling this discipline across a team.